# Cheap to write, *expensive to trust.*

*The effects of AI-assisted software engineering on code quality and security — two industry challenges and a like-for-like Catan case study.*

Dao Trong-Nghia
TEAM 08

Mata Marco
TEAM 08

Galindo Lopez Pablo Emiliano
TEAM 08

Jan Kott
TEAM 08

# Adoption is near universal. *Verification is not.*

AI made writing code cheap. Checking it is not. The tools have gotten better; the share of AI-generated code with high-severity vulnerabilities has not.

**45%**

of AI-generated code carries at least one high-severity vulnerability, across 100+ LLMs evaluated. Has not significantly improved as tools mature.

**90%**

of software professionals now use AI assistants — median two hours a day — and DORA reports a *negative* correlation with software delivery stability.

# Two distinct problems, one root.

Teams ship more code than reviewer capacity can absorb. The bottleneck is moving from writing to reviewing — and the bad code reaches production along the way.

CHALLENGE 1

## Quality & security drift

What AI puts into the codebase is buggier and more duplicated than what it replaced.

CHALLENGE 2

## Review cannot keep up

When bad code is produced, the people meant to catch it are buried in volume. Open-source maintainers are hit hardest.

For each challenge we surface one industry failure and one industry success — then test the drift claim ourselves in Task 2.

# Prevention exists. *It just doesn't run at AI speed.*

FAILURE CASE

## Clawdbot / OpenClaw

A viral open-source AI agent project. During a forced rename, crypto scammers seized the GitHub organisation in **ten seconds**. Running instances appeared on Shodan with plaintext credentials.

Every prevention tool that should have caught this already existed — none operated at the speed required.

WORKING CASE

## GitHub Dependabot at scale

Across 2.66 M repositories with Dependabot enabled, automated dependency-vulnerability fix times collapsed dramatically.

| BEFORE | | AFTER |
|---|---|---|
| 200–300 days | → | 30–50 days |

Automation closes a security gap at scale — but only where the gap is shaped like dependency hygiene.

# Reviewer asymmetry: *12× and rising.*

FAILURE CASE

## curl bug bounty closure

curl's bug-bounty programme was flooded with AI-generated reports that sounded professional and were almost all wrong. Stricter rules and examples of what not to submit did not help.

The maintainer shut the programme down entirely.

WORKING CASE

## Google internal review tooling

**97%** engineer satisfaction. An AutoCommenter trained on ~**3 B examples** flags best-practice violations; a critic in the Jules assistant critiques AI code as it's produced.

A working playbook — that depends on Google's scale. A one-person project can't replicate it.

RESEARCH QUESTION

# Does the quality and security debt reported in the literature show up on a *like-for-like* AI-versus-human implementation of the same specification under one static analyser?

SPECIFICATION

**PM1 Project 3 — Settlers of Catan**, a ZHAW first-semester Java assignment: base game, a cities extension via inheritance, and a random-victim robber variant. 30 points total.

TWO IMPLEMENTATIONS

**Human:** frozen snapshot of `boostvolt/zhaw-catan`, used with permission.
**AI:** Claude Opus from a consolidated English spec — same scope, same `ch.zhaw.hexboard`, same Java subset.

ANALYSIS

One static analyser applied to both Maven builds. Compare findings on the same denominator. Directly tests Challenge 1; addresses Challenge 2 via the production-versus-verification asymmetry the comparison reveals.

# Six candidates. *One fits two static Java snapshots.*

| TOOL | SURFACE | FIT FOR CATAN | VERDICT |
|---|---|---|---|
| **Dependabot** | Manifests, lockfiles | Tiny dependency surface; nothing to flag | Discarded |
| **GHAS / CodeQL** | Source code | Strong on Java, but security-only — misses maintainability and reliability signal | Runner-up |
| **Secret Protection** | Commits, history | Offline game, no secrets | Discarded |
| **Copilot code review** | Pull request diff | No active pull request | Discarded |
| **Dependency Review** | PR manifest diff | No PR and no manifest change | Discarded |
| **SonarQube** | Full Java Maven project | Bugs, reliability, maintainability, and hotspots in a single scan — no PR or dependency event required | **Selected** |

Node.js benchmark confirmed each tool works in its native habitat; SonarQube won on *broader signal* because the study is about quality debt, not just vulnerabilities.

# Quality drift is visible. *Security delta is zero.*

## STRUCTURAL COMPARISON

| | AI | HUMAN |
|---|---|---|
| Main LOC | 1,108 | 2,718 |
| Main files | 9 | 17 |
| Test LOC | 260 | 2,060 |
| Test / Main ratio | 0.23 | 0.76 |

AI omits a Road class and the abstract `Structure` base; concentrates 39% of main source in one `SiedlerGame`. Produced in ~**10 minutes** against weeks for the human side.

## SONARQUBE · SCAN 2026-05-01

| | AI | HUMAN |
|---|---|---|
| Lines of code | 1.6 k | 2.0 k |
| Security rating | A · 0 | A · 0 |
| Reliability rating | D · 3 | C · 1 |
| Maintainability | A · 16 | A · 9 |
| Security hotspots | 3 | 1 |

Per kLOC: reliability **1.9** vs **0.5**; maintainability **10.0** vs **4.5**. Both pass the quality gate.

# The literature claim is *partially* supported.

SUPPORTED

## Reliability & maintainability

AI scores worse even on the per-kLOC denominator that favours the smaller codebase. Findings concentrate in oversized methods inside `SiedlerGame` — cognitive complexity up to **49** against an allowed 15.

NOT SUPPORTED

## Direct security

Both projects are effectively blank — but that's a property of the input. Small offline Java with tiny dependency surface and no secret-handling path gives vulnerability tooling little to bite on. Four hotspots, all benign `Random` usage.

CAVEAT

## Not perfectly equivalent

Test-to-main ratio 0.23 vs 0.76; AI omits the Road class and the abstract base. Density readings matter more than absolute counts. Static analysis does not validate game-rule correctness.

# Where the engineering work actually goes.

01 Production cost has decoupled from verification cost.

Ten minutes for 1,108 main lines against weeks of human work — the cheap thing got cheaper; the expensive thing did not.

02 Static analysis catches quality drift where theory predicts it.

Cognitive complexity 49 in `SiedlerGame`, repeated god-class smells — visible. Rule-correctness stays outside the scanner.

03 A clean security rating depends on scan surface.

A small offline project with no secrets, no network, and minimal dependencies gives vulnerability tooling little to detect. Don't read absence as safety.

04 Prevention tooling alone is not enough.

Every failure case had relevant tooling somewhere in the ecosystem — none operated at the speed or scale the most exposed projects needed.

05 AI shifts engineering work; it does not remove it.

Less effort for the first draft; more discipline in review, validation, static analysis, security assessment, and architectural judgment.

# Code Showdown — *human vs AI.*

Two anonymous Java solutions to the same problem. Same algorithm, same Big-O. One written by a human, one by an AI. You decide which is which — and which you would merge.

01 **Read both snippets side-by-side.**

Same Valid Sudoku problem from LeetCode. Same time and space complexity. The differences are purely stylistic.

02 **Vote: which would you merge into production?**

Think readability, maintainability, and review cost — not correctness. Both pass.

03 **Vote: which one was written by the AI?**

We reveal the answer, compare votes, and discuss what gave it away.

# The task — Valid Sudoku.

From LeetCode. Determine whether a 9 × 9 Sudoku board is valid. Only the filled cells need to be validated.

Rule 01

### Rows

Each row must contain the digits 1–9 without repetition.

Rule 02

### Columns

Each column must contain the digits 1–9 without repetition.

Rule 03

### 3 × 3 sub-boxes

Each of the nine 3 × 3 sub-boxes must contain the digits 1–9 without repetition.

A valid board is not necessarily solvable. Empty cells are denoted by `'.'` and are skipped during validation.

# Snippet A vs Snippet B. *Both anonymous.*

## Snippet A

Anonymous · Java

```java
boolean[][] rows  = new boolean[9][9];
boolean[][] cols  = new boolean[9][9];
boolean[][] boxes = new boolean[9][9];

for (int i = 0; i < 9; i++) {
  for (int j = 0; j < 9; j++) {
    if (board[i][j] != '.') {
      int num = board[i][j] - '1';
      int boxIndex = (i / 3) * 3 + (j / 3);

      if (rows[i][num] || cols[j][num]
          || boxes[boxIndex][num]) {
        return false;
      }
      rows[i][num] = cols[j][num]
          = boxes[boxIndex][num] = true;
    }
  }
}
return true;
```

## Snippet B

Anonymous · Java

```java
// Track seen digits per row/col/3×3 box
boolean[][] rows  = new boolean[9][9];
boolean[][] cols  = new boolean[9][9];
boolean[][] boxes = new boolean[9][9];

for (int r = 0; r < 9; r++) {
  for (int c = 0; c < 9; c++) {
    if (board[r][c] == '.') {
      continue; // skip empty cells
    }
    // Convert '1'-'9' to index 0-8
    int val    = board[r][c] - '1';
    int boxIdx = (r / 3) * 3 + (c / 3);

    // Already seen in row, col, or box?
    if (rows[r][val] || cols[c][val]
        || boxes[boxIdx][val]) {
      return false;
    }
    // Mark as seen
    rows[r][val]      = true;
    cols[c][val]      = true;
    boxes[boxIdx][val] = true;
  }
}
return true;
```

# Two questions for the room.

QUESTION 01

## Which would you merge?

Imagine this is a pull request on your team. Which snippet do you approve for production? Think readability, future maintainers, and review effort — not correctness.

A

B

QUESTION 02

## Which one is the AI?

What stylistic tells gave it away? Comments, variable names, control flow, indentation, repetition? We reveal, compare to your first vote, and debrief together.

A

B

TEAM 08 · ASE2 · ZHAW

AI is a *drafting accelerator*, not a replacement for software-engineering responsibility.

# *Questions?*

DAO TRONG-NGHIA · MATA MARCO · GALINDO LOPEZ PABLO EMILIANO · JAN KOTT